

When Does Population-Mobility OD Injection Help Traffic Forecasting? A Capacity-Gap Account and a Validation-Gated Remedy

Sumin Han

Abstract—Road traffic is a downstream consequence of population movement, so forecasted origin–destination (OD) flow, routed onto the road network, should carry predictive information a traffic-only model lacks. Under 30-fold rolling-origin validation on Seoul’s city-wide population-flow data (500 administrative units, 2023–2026, volume on 121 sensors and speed on 396), injecting a routed OD forecast through a learned per-node, per-timestep gate reduces a lightweight graph-recurrent model’s MSE by 6.5–8.3% ($p < 10^{-5}$). Applied uniformly to ten published architectures spanning 32k–774k parameters, however, the same mechanism helps only the two smallest and is significantly *harmful* above 10^5 parameters (Spearman $\rho = 0.89$, $p = 5 \times 10^{-4}$) – a capacity-gap pattern a within-architecture sweep and three further public benchmarks (PEMS-BAY, METR-LA, PeMSD7) both reproduce. This is not where the paper stops: a per-sensor selector, chosen from each fold’s validation split alone and requiring no retraining, recovers significant improvement in eighteen of twenty architecture/task combinations – including seven where uniform injection was significantly harmful – and nearly doubles the one case already favorable, turning a mechanism that fails to generalize into a practical, checkpoint-level fix that does. We report this alongside the checks needed to trust it rather than the checks that would make it look better: a calendar-climatology control shows most, though not all, of the lightweight-model gain is climatological rather than OD-specific, and a further decomposition shows the genuine, non-calendar share is real but modest (34% of the average gain on its own); and a seed-control negative experiment shows the selector’s own apparent benefit is, in three of the twenty combinations, not distinguishable from validation-noise exploitation once measured and subtracted – specifically the three combinations uniform injection harmed the most – a caveat we quantify and report rather than omit.

Index Terms—Traffic prediction, origin–destination flow, road network routing, spatio-temporal knowledge injection, urban computing, empirical study, graph neural networks.



1 INTRODUCTION

Accurate short-term forecasting of road traffic volume and speed underlies congestion management, route guidance, signal control, and intelligent transportation service operation. Over the last decade the dominant modeling paradigm has shifted from classical statistical time-series methods [1], [2] to deep spatiotemporal graph models that treat traffic forecasting as a purely *endogenous* problem: a model observes past sensor readings on a road-network graph and extrapolates forward, using graph convolution [3]–[5], attention [6]–[8], or learned/adaptive graph structure [9]–[11] to capture spatial correlation among sensors. This framing discards information that is causally *upstream* of traffic itself: the origin–destination (OD) movement of the population that produces that traffic. A commuter’s decision to travel from home to work precedes, by minutes to hours, the vehicle that decision puts on a specific sequence of road links. If a meaningful fraction of vehicular trips can be anticipated from aggregate population-mobility forecasts *before* they manifest as sensor readings, and correctly attributed to the specific road links they will physically cross, injecting them should provide predictive signal a traffic-only model cannot access from its own inputs alone.

This is not merely theoretical: municipal and national transportation authorities increasingly have access to, or can procure, aggregate population-mobility products (mobile-carrier location records, credit-card transaction geodata, ride-hailing logs) at update cadences fast enough to be genuinely predictive rather than merely descriptive. If such a signal reliably improves traffic forecasting once properly attributed to the road network, it is a comparatively cheap lever: injecting an auxiliary signal into an already-deployed lightweight model requires no re-architecting around a larger, more capable spatiotemporal model – itself a nontrivial engineering and retraining investment that may not be feasible for smaller municipal deployments on constrained hardware. Understanding precisely when this lever pays off, and when it does not, therefore has direct practical as well as scientific value, motivating the systematic, capacity-conditioned investigation this paper undertakes rather than a report of a single positive result on a single architecture.

A large and growing fraction of the traffic-forecasting literature reports positive results for a proposed technique on a small number of architectures and datasets, with comparatively little attention paid to the boundary conditions under which the same technique would *not* help – information at least as valuable to a practitioner deciding whether to adopt a technique. This paper is deliberately structured around both halves of that picture: Sections 6–7 establish a genuine, rigorously validated positive effect, and Section 8 characterizes, rather than suppresses, the specific

- Sumin Han is with the Korea Science Academy of KAIST, Busan, Republic of Korea (e-mail: suminhan@ksa.kaist.ac.kr).
- Code, analysis scripts, and result summaries: <https://github.com/SuminHan/traffic-od-injection-capacity-gap>

and reproducible conditions under which that same effect vanishes or reverses. We view the willingness to report and mechanistically explain a null and a significantly negative result, not only the configurations that happened to work, as itself part of this paper’s contribution.

Prior work incorporating population or regional context into road-level traffic prediction has typically done so through a *learned*, implicit mapping – e.g., [12] embeds region-level knowledge (LTE traces, points of interest, satellite imagery) via attention the network learns to associate with road-level representations, on Seoul traffic speed data as we also use. This is powerful, but the association is learned end-to-end and not grounded in any explicit statement of *why* a region’s population should influence a given road link. This paper instead tests an *explicit, mechanistic* channel: a road-hierarchy-weighted shortest-path assignment that routes forecasted OD flow onto the specific links it would physically cross, so the injected signal at a sensor has a concrete, verifiable graph-theoretic justification rather than a purely learned association that could reflect confounding.

We validate this with a level of statistical rigor uncommon in this literature: rolling-origin walk-forward cross-validation with up to 30 folds (12-month train / 1-month validation / 1-month test, 1-month stride, so every reported number is genuinely out-of-sample), paired parametric and non-parametric significance testing on every fold, and independent replication across two sensor modalities (volume, speed) and two date ranges. On our own lightweight graph-recurrent architecture, the result is unambiguous and strongly significant in every fold set tested.

This raises a natural follow-up question: is this a general property of the injection mechanism, or an artifact of using a simple base architecture that has not yet learned to extract, on its own, the regularities a stronger model would already capture? We apply the identical idea – realized five different architectural ways – to ten published or reimplemented architectures spanning a $24\times$ range in parameter count: STID [13], GMAN [7], STGCN [3], PDFormer [14], STAEformer [8], MTGNN [11], DCRNN [4], GTS [10], Graph WaveNet [5] and AGCRN [9] – including graph-free attention-only designs. Ordered by parameter count, the outcome is close to monotone: injection is significantly *favorable* only for the two smallest architectures (STID, GMAN, both on volume only), null near 10^5 parameters, and significantly *harmful* for several of the largest. Across the twenty (architecture, task) points the effect correlates with log parameters at Spearman $\rho = 0.67$ ($p = 1.2 \times 10^{-3}$), and a capacity sweep *within* STID reproduces the same decay with architecture, recipe and signal held fixed. We treat this null-to-negative pattern as the paper’s second central contribution and diagnose it directly, using (i) an oracle experiment injecting the *true*, already-observed OD instead of a forecast, ruling out forecast inaccuracy as the explanation, and (ii) a per-sample analysis of exactly which hours the signal helps, pointing to a specific, mechanistically interpretable explanation: capacity-gap redundancy.

The capacity-gap account explains why a fixed, architecture-internal fusion mechanism fails to generalize in aggregate, but leaves open whether the signal carries real, exploitable value at the level of individual sensors within these stronger architectures. We test this with a third ex-

periment: a per-sensor selector, chosen independently each fold from only that fold’s validation split, deciding whether the injected or the plain checkpoint serves each sensor at test time. This selective, validation-gated injection recovers a statistically significant improvement in eighteen of twenty (architecture, task) combinations with a full 30-fold sweep – including seven where uniform injection was significantly harmful – and nearly doubles GMAN’s already-favorable volume result. This general, architecture-agnostic recipe (Section 7.1) is this paper’s central practical contribution: the capacity-gap boundary applies to what an architecture’s own fusion mechanism can learn to exploit from a fixed injection scheme, not to the presence of real, sensor-level value in the signal – and that value is recoverable, cheaply and without retraining, regardless of architecture.

Our contributions are:

- A general, architecture-agnostic recipe – selective, validation-gated injection – that recovers auxiliary-signal value a fixed fusion mechanism fails to extract, choosing per sensor from validation data alone whether to trust an injected or a plain checkpoint, with no architectural change and no retraining beyond the two checkpoints already needed for the underlying comparison. This recovers a significant favorable effect in eighteen of twenty (architecture, task) combinations we test, including seven where uniform injection was significantly harmful; a false-discovery-rate-corrected per-sensor analysis and a random-selection negative control together confirm the effect is real heterogeneity, not sampling noise or generic ensembling – and a seed-control check reports honestly where it is not (three of the twenty combinations, specifically the three most severely harmed by uniform injection), rather than presenting only the favorable cases.
- The negative result this recipe is built to fix: a rigorously validated (30-fold, dual-modality) demonstration that road-network-routed OD injection significantly improves traffic forecasting for a lightweight base model, closing a substantial share of the gap to a considerably stronger published architecture using only a cheap auxiliary signal – but a systematic test of the same idea, realized five different architectural ways, across eight larger published architectures shows it does *not* generalize, and is significantly harmful for several of the largest; one architecturally independent exception (GMAN, the smallest published architecture tested) reinforces rather than undermines the pattern.
- A *capacity-gap* explanation for why the fixed mechanism fails and the selective one does not – the signal is redundant once a model is expressive enough to infer it implicitly – supported by an oracle experiment that rules out forecast inaccuracy as the cause, a per-sample diagnostic localizing where the signal’s value concentrates, and two independent capacity sweeps (within our own architecture and within a published one) showing capacity and injection act as substitutes, not independent sources of accuracy.
- Supporting evidence for external validity:

reimplementation-fidelity checks against published benchmark numbers, and a fusion-mechanism ablation and verified routing pipeline ruling out implementation shortcuts as alternative explanations for our results.

Sections 2–5 review related work, formalize the problem, and describe our methodology and experimental setup. Section 6 presents the main result on our lightweight model. Section 7 shows this fails to generalize to published architectures, leads with the practical fix – selective, validation-gated injection (Section 7.1) – then gives the deeper diagnostic material explaining *why*. Section 8 discusses limitations, and Section 9 concludes. Supporting material not essential to the main argument is in the supplementary material (footnote, page 1): a fusion-architecture ablation, an adaptive-threshold refinement of the selective rule, cross-dataset replication, a qualitative case study, two smaller exploratory findings, and the reimplementation-fidelity check referenced in Section 7.

2 RELATED WORK

2.1 Classical and Early Traffic Forecasting

Traffic forecasting has a long history predating deep learning. Autoregressive integrated moving average (ARIMA) models and their seasonal variants [1], and Kalman filtering [2] and its extensions, model traffic as a univariate or low-dimensional linear time series per sensor. These methods are computationally cheap and interpretable but cannot capture the nonlinear spatial dependencies among sensors that dominate real road networks, particularly the propagation of congestion along and across roads. Extensions that add a spatial component – e.g. vector autoregression across sensors, or Kalman filters with a spatially structured transition matrix – partially address this, but the linear-Gaussian assumptions underlying both families still limit their ability to model the sharp, nonlinear regime changes (free flow to congestion, and back) that characterize urban traffic, which is precisely the failure mode that motivated the shift to deep, nonlinear spatiotemporal models reviewed next. We do not re-run ARIMA or Kalman filtering as active baselines in this paper’s experiments, since prior work in this space has repeatedly and consistently shown deep spatiotemporal models to dominate them on comparable road-sensor benchmarks [4], [5]; we cite them here for historical completeness and because the capacity-gap phenomenon central to this paper’s findings (Section 8) is, in a loose sense, a modern echo of exactly the representational-capacity limitation that motivated the original move away from these classical linear methods.

2.2 Deep Spatiotemporal Graph Models for Traffic

The now-dominant paradigm represents the sensor network as a graph and combines graph-structured spatial mixing with temporal sequence modeling. STGCN [3] was among the first to combine graph convolution with gated temporal convolution. DCRNN [4] instead models traffic diffusion as a bidirectional random-walk process on a directed graph, embedded inside a recurrent encoder-decoder; we adopt it

as one of our three published-baseline architectures. ASTGCN and ASTGNN [6], [15] add spatial and temporal attention to weight the contribution of different neighbors and time steps adaptively – the latter, published in this journal, further models heterogeneity across nodes and time. AGCRN [9] and GTS [10] replace the fixed, geography-derived adjacency with a graph that is itself learned end-to-end from node embeddings (the latter via a Gumbel-softmax-sampled discrete structure initialized at a physical prior); we adopt GTS as our third published-baseline architecture. These learned node embeddings are conceptually related to, though jointly optimized with the forecasting objective rather than pre-trained separately from, classical graph-representation-learning methods such as node2vec [16], which learns node embeddings from biased second-order random walks over a fixed graph; we explore a node2vec-initialized variant of this idea directly in our own experiments (Appendix E, supplementary material), finding that a frozen node2vec initialization consistently hurts accuracy while fine-tuning from that initialization consistently helps relative to a random initialization – a small result in its own right, but one that echoes this paper’s broader theme of pre-existing structural knowledge acting as a useful *prior* rather than a fixed substitute for what the model must ultimately learn end-to-end. Graph WaveNet [5], our second baseline, instead learns a dense adaptive adjacency from two node-embedding matrices and combines it with dilated causal convolutions, avoiding recurrence entirely for computational efficiency. STSGCN [17] synchronizes local spatiotemporal correlation directly rather than modeling space and time as separable factors. MTGNN [11] generalizes graph-based forecasting to arbitrary multivariate time series without assuming a given graph at all. GMAN [7] relies purely on stacked spatial and temporal attention bridged by a transform-attention module. More recent architectures push toward efficiency and long-range dependencies at scale: D2STGNN [18] decouples diffusion and inherent signal components; PDFormer [14] models propagation delay explicitly with a long-range transformer; STAEformer [8] shows that a carefully designed spatiotemporal adaptive embedding lets a vanilla Transformer match specialized architectures; BigST [19] targets linear-complexity inference over very large road networks via a pre-computable long-sequence feature extractor. T-GCN [20] and the crowd-flow-oriented ST-ResNet [21] represent two further influential lines combining graph or convolutional spatial modeling with recurrent or residual temporal modeling. We adopt DCRNN, Graph WaveNet, GTS, STAEformer, and GMAN as published-baseline architectures throughout Section 8, spanning several qualitatively different approaches to spatial structure (fixed diffusion, learned adaptive adjacency, learned discrete graph structure, and attention-only graph-free modeling). An initial GMAN reimplementation failed to reach a competitive training loss under standard settings; the issue was that GMAN’s pure self-attention over the lookback window has no inherent sequential inductive bias (unlike DCRNN’s recurrence or Graph WaveNet’s causal dilated convolutions), so it never learned a notion of recency without an explicit positional signal. Adding standard sinusoidal positional encoding over the lookback axis resolved this, and GMAN trains stably and is included throughout

Section 8 like every other baseline.

2.3 Population/Regional-Knowledge-Aware Traffic Prediction

Prior work has explored injecting exogenous population or regional context into road-level traffic models. Most directly related is [12], discussed above, which embeds real-time region-level knowledge via a *learned* bipartite spatial transform attention to convert region-level to road-level representations, evaluated on Seoul traffic speed and living-population data in three downtown districts. Our approach differs in kind, not just degree: instead of a learned implicit region-to-road mapping, we compute an *explicit* shortest-path assignment over the actual road network, so a given sensor only receives signal from OD pairs whose routed path provably crosses that sensor’s link. Our own prior work [22] established the foundational floating-population embedding this line of work builds on, applied to traffic prediction without explicit road-network routing.

2.4 Knowledge Adaptation Between Data-Rich and Data-Sparse Sources

Beyond traffic specifically, knowledge adaptation from a data-rich source to a data-sparse target is a recurring pattern in urban prediction. Within a single city, [23] transfers demand- forecasting knowledge from a station-intensive public-transit mode (bus) to station-sparse modes (train, light rail, ferry) via a memory-augmented multi-task network with a learned *boost/eliminate* dual-vector gate, reporting that naive feature concatenation of the two sources actively *hurts* accuracy relative to the target-only model – a finding we independently reproduce in a different setting in Appendix A (supplementary material). We adapt this dual-gate mechanism as one of the four injection realizations tested in Section 8. Across cities, RegionTrans [24] and its inter-city region-matching extension [25] transfer spatiotemporal prediction knowledge from a data-rich source city to a data-scarce target city; CrossTRES [26] selectively re-weights source-city regions by relevance before transfer; and, in a different urban sensing domain, FLORAL [27] adapts semantically related dictionaries across modalities for air-quality prediction. Across this broader literature, reported successes share a consistent structural signature: the target is a genuinely data- or capacity-limited learner – few stations, a short training history, a data-poor city, or, in our terms, a lightweight model. This pattern is consistent with, and directly motivates, the capacity-gap explanation we develop in Section 8.

2.5 Foundation and Universal Models for Spatiotemporal Prediction

A separate and rapidly growing line of work pursues generalization *across* datasets, cities, and even modalities rather than richer per-dataset signal. UniST [28] pre-trains a single Transformer across diverse spatiotemporal domains via masked-token modeling and adapts to new scenarios with knowledge-guided prompts in few-/zero-shot settings. GPT-ST [29] and STEP [30] similarly pursue generative or predictive pre-training for spatiotemporal graphs. Most

recently, large language models have been adapted directly to spatiotemporal forecasting: UrbanGPT [31] couples a spatiotemporal dependency encoder with an instruction-tuned LLM for zero-shot urban prediction; ST-LLM [32] tokenizes per-location time steps with spatial-temporal embeddings and partially freezes LLM attention to adapt it to traffic data; and OpenCity [33] combines Transformer and graph-neural-network components into an open foundation model with promising zero-shot and scaling behavior across cities. This direction is orthogonal to ours – these works study generalization of a shared backbone across datasets and cities, whereas we study a single-city, cross-modality knowledge-injection mechanism and characterize precisely when and why it helps, which is a question that remains unanswered regardless of how the underlying backbone is pre-trained or shared.

3 PRELIMINARIES AND PROBLEM DEFINITION

Road network. The capital-region road network is represented as a directed graph $\mathcal{G} = (\mathcal{V}_r, \mathcal{E}_r)$ with 315,048 nodes and 430,675 directed edges, built from MOCT LINK/NODE shapefiles restricted to the capital-region bounding box. Each edge $e = (u, v)$ has weight

$$w_e = \ell_e \cdot \rho_e^\alpha, \quad (1)$$

where ℓ_e is physical edge length in meters, $\rho_e \in (0, 1]$ is a road-hierarchy discount factor mapping the seven MOCT road-rank categories to values from ≈ 0.50 (local road) up to 1.00 (motorway), and $\alpha = 0.9$ controls the strength of the hierarchy preference relative to raw distance in shortest-path routing (higher α more strongly prefers major roads even at the cost of extra distance).

OD movement. $\mathcal{X}_{od} \in \mathbb{R}^{T \times N \times N}$ is the hourly origin–destination population flow between $N=500$ administrative (dong-level and, outside Seoul proper, gu/si-level) units for T hours spanning 2023-01 through 2026-07/08, derived from mobile-phone location records and released as a public living-mobility dataset.

Traffic sensors. Two independent sensor sets from Seoul’s TOPIS system: $S_v=121$ point volume sensors (vehicles/hour) and $S_s=396$ link-level speed sensors (km/h), each snapped to the nearest road-graph edge by minimizing Euclidean distance in a projected coordinate system (mean snap distance 46m and 25m respectively, reflecting that speed sensors already carry native link geometry while volume sensors are point counters).

Forecasting task. Given $P=12$ hours of history for every sensor, forecast the next $Q=12$ hours: $\hat{X}_{t+1:t+Q} = \mathcal{F}_\theta(X_{t-P+1:t})$. We report MSE, RMSE, MAE, and MAPE, with a near-zero-denominator guard on MAPE (≥ 10 vehicles/h for volume, ≥ 1 km/h for speed) chosen per-modality because near-zero speed is a genuine congestion state that should not be treated as a sparsity artifact, whereas near-zero volume typically is.

Rolling-origin evaluation. All results use walk-forward validation: 12 months train / 1 month validation / 1 month test, stride 1 month, giving 23 folds over the sensors’ native 2023–2025 range and 30 folds over an extended 2023–2026.07 (volume) / 2026.08 (OD only) range. The extended range was recovered by discovering that raw monthly source files

were already available through mid-2026 but a shared upstream aggregation script had a hard-coded date range and had silently stopped incorporating them; we re-aggregated the missing months directly from the raw files into our own project-local cache rather than modifying the shared upstream pipeline.

Why multiple fold counts appear in this paper. The extended range was recovered partway through this study, so different results here were computed at different points along that migration and are reported at different fold counts rather than silently mixed within one comparison. Table 4 (our own model’s central OD-injection result) and the mechanism-(i) rows of Table 10 (the published-baseline generalization study) are both on the full extended 30-fold range. The capacity sweep (Section 6) is a single-fold (fold 0) measurement, not rerun across folds because it varies model capacity rather than testing significance. The remaining diagnostics – the per-sample anomaly analysis of Section 7.4, the oracle and mechanisms (ii)–(v) of Section 8 – were run before the range was extended and are reported at their original, smaller native-range fold counts (each row of Table 10 states its own count) as supplementary evidence, not rerun at 30 folds given the GPU cost and their secondary role. We never mix folds from different ranges within a single significance test.

Formal statement of the injection question. Let $\mathcal{F}_\theta$ be a traffic forecaster taking only past traffic as input, and $\mathcal{F}'_{\theta,\phi}$ the same architecture augmented with a routed OD signal $od_{t-P+1:t+Q}$ through a fusion module parameterized by ϕ (Section 4.3). We are not testing whether ϕ can be fit to reduce training loss – with enough capacity it trivially can, since od is available at both train and test time and a sufficiently expressive ϕ can always learn to ignore it if it is unhelpful – but whether the resulting *test-set* generalization of $\mathcal{F}'$ is significantly better than $\mathcal{F}$ under a paired, walk-forward-consistent evaluation protocol, and whether that answer depends on the representational capacity of $\mathcal{F}_\theta$ itself. This framing – a generalization question conditioned on base-model capacity, rather than a fixed claim that “feature X helps model Y ” – is what motivates testing the same od signal against five architectures of differing capacity (Sections 6 and 8).

4 METHODOLOGY

4.1 OD Movement Forecasting

A gated-fusion graph encoder-decoder over the 500-node OD graph (*HybridOD*) forecasts per-origin hourly outflow one step ahead, conditioned on calendar features (hour-of-day and day-of-week sinusoidal encodings, and Korean public-holiday indicators for the target day and its immediate neighbors). Its architecture – a node/timestep-wise gated fusion of a dynamic-edge-weight graph GRU branch, chosen after a dedicated architecture search across 14 of a planned 93 rolling-origin folds showed the design had converged to a stable, well-performing configuration – was frozen at a single fold-0 checkpoint to produce a single, internally consistent OD forecast used identically across every downstream traffic-forecasting experiment in this paper, avoiding the confound of a changing upstream forecast quality across folds.

TABLE 1
Notation used throughout the paper.

Symbol	Meaning
$\mathcal{G}, \mathcal{V}_r, \mathcal{E}_r$	Road network graph, nodes, directed edges
$w_e, \ell_e, \rho_e, \alpha$	Edge weight, length, hierarchy discount, its exponent
$\mathcal{X}_{od}$	Hourly $N \times N$ OD movement tensor
N	Number of OD administrative units (500)
S, S_v, S_s	Sensor count (general, volume = 121, speed = 396)
P, Q	Input / forecast horizon length (12, 12 hours)
W	Static $500 \times S$ routing-weight matrix
$\text{share}(o \rightarrow d)$	Historical destination-share distribution of origin o
od_t, routed_t	Per-origin outflow forecast / routed sensor signal at hour t
$h_t^{\text{traffic}}, h_t^{\text{od}}$	Traffic-branch / OD-branch hidden state at step t
$\beta_t, g(\cdot)$	Per-node/per-step fusion gate, its (MLP) parameterization
θ, ϕ	Base-model / fusion-module parameters

Concretely, HybridOD represents the $N=500$ administrative units as nodes of a graph whose edges are the units’ historical (training-period) OD flow magnitudes, so that two units with a large mutual population flow are treated as strongly connected regardless of their physical geographic distance – e.g. two dense, distant business districts connected by a fast subway line will have a strong graph edge despite being far apart on the road network, whereas two adjacent but poorly connected neighborhoods will not. A dynamic-edge-weight graph GRU cell aggregates each node’s neighbors at every hour using edge weights that are themselves modulated by the current hour’s calendar features (so that, for instance, the effective strength of a commute-heavy edge can differ between a weekday morning and a weekend afternoon), and the resulting per-node hidden state is fused, via the same per-node gated-fusion pattern used in the downstream traffic model (Section 4.3), with a purely calendar-conditioned branch that captures each node’s typical (day-of-week, hour-of-day, holiday-adjusted) baseline outflow independently of the graph-propagated signal. This fusion pattern – letting the model learn, per node and per hour, how much to trust graph-propagated neighbor information versus a node’s own typical baseline – was itself the product of the 14-fold architecture search referenced above, in which several simpler alternatives (a pure graph GRU with no calendar branch; a pure calendar-baseline model with no graph propagation at all) were both found to underperform the fused design, consistent with the broader pattern this paper documents at the traffic-forecasting level: neither a purely structural nor a purely statistical signal alone is sufficient, and the value lies specifically in a model that can learn to combine both conditionally.

4.2 Road-Network Routing

For each origin unit o , Dijkstra’s algorithm on $\mathcal{G}$ (edge weights as in Section 3) gives a shortest-path tree to every other unit; this is run once per origin (not per OD pair), making the full sweep over all $N=500$ origins tractable (≈ 120 seconds on the full 315k-node graph). Each origin’s

historical (2023 training-period) destination-share distribution share($o \rightarrow d$) is computed from the ground-truth OD tensor. A static routing-weight matrix $W \in \mathbb{R}^{500 \times S}$ is built once:

$$W[o, s] = \sum_d \text{share}(o \rightarrow d) \cdot \mathbb{I}[\text{path}(o, d) \text{ crosses sensor } s\text{'s link}] \quad (2)$$

Applying W to *any* hour's per-origin outflow forecast via a single matrix multiplication $\text{routed}_t = \text{outflow}_t \cdot W$ yields a routed sensor-level signal for that hour. W is a purely structural object – built once from the road network and the 2023 training-period destination-share distribution – and is reused unchanged across every rolling-origin fold, since it encodes road-network topology and typical destination preference rather than any particular hour's traffic. This is what is injected, not a naive same-district outflow value pasted onto the nearest sensor, which we verified in early, unsuccessful pilot experiments carries no useful signal: without the graph-routing step, a dong's raw outflow number is essentially uncorrelated with any specific downstream sensor's traffic, since it says nothing about *which* roads that outflow actually travels along. We verified that the routing step is genuinely computed, rather than a stand-in heuristic, via an interactive visualization that reconstructs and renders the actual shortest-path polylines together with each sensor's top- K contributing origin–destination pairs and the administrative-district boundaries of those origins and destinations.

Relation to classical traffic assignment. Equation (2) is, in the terminology of the transportation literature, a deterministic *all-or-nothing* network loading: each origin's demand is assigned entirely to shortest paths, with no congestion feedback and no dispersion in route choice. It therefore satisfies neither of Wardrop's equilibrium principles [34] and does not solve the convex program that makes user equilibrium computable [35], [36]. This is deliberate, and we state the tradeoff plainly because a transportation-side reader will notice it immediately. An equilibrium assignment would require link performance functions, a congestion feedback loop, and an iterative solve for every one of the 31,392 hours in the study range, and – more fundamentally – would make the injected signal a function of the very congestion state the downstream model is trying to predict, reintroducing exactly the circularity that a forward-looking demand proxy is meant to avoid. What we need from W is not an accurate flow prediction but a stable structural map from origins to the sensors their outflow plausibly traverses; the learned gate of Section 4.3 is what decides how much to trust the resulting signal, hour by hour and sensor by sensor.

Consistent with this framing, we additionally explored, but do not adopt as the primary mechanism after negative results (Appendix A, supplementary material), a Monte Carlo perturbed-Dijkstra stochastic multi-path variant of W (five independent passes per origin with log-normal multiplicative noise on edge weights, crediting a sensor fractionally by how often it was crossed across passes rather than only along the single strict shortest path). This is a crude stand-in for stochastic network loading in the sense of Dial's path-enumeration-free logit assignment [37], intended to better approximate real drivers' route choice; in practice it performed no better, and in one tested case

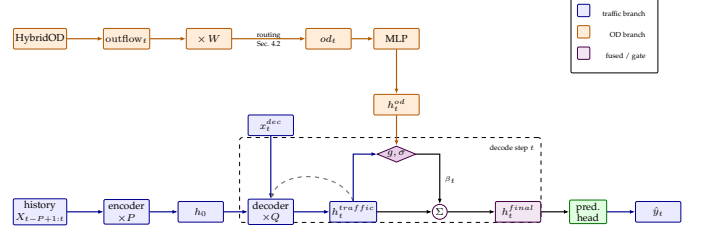


Fig. 1. End-to-end gated-fusion injection architecture (Section 4.3). *Upper left*: the frozen HybridOD forecast is routed onto the road network via the static matrix W (Section 4.2) to produce od_t . *Main flow*: the traffic encoder-decoder consumes recent history; at each decode step the routed signal is projected to h_t^{od} and combined with the traffic branch's $h_t^{traffic}$ through a per-node, per-timestep gate β_t (equation in text). Only the pure $h_t^{traffic}$ (not the fused h_t^{final}) is carried into the next step's recurrence.

slightly worse, than the single-shortest-path W – suggesting that at the spatial and temporal granularity of this signal, route-choice dispersion is not the binding limitation.

4.3 Gated Fusion Injection (Primary Mechanism)

The traffic model is a graph-recurrent encoder-decoder over a k -nearest-neighbor, Gaussian-kernel-weighted sensor graph (*WeightedGraphGRUCell*), with the kernel bandwidth set adaptively to the median nearest-neighbor distance among sensors. At every decode step t , the routed OD signal is projected through a small MLP, $h_t^{od} = \text{MLP}(od_t)$, and fused with the traffic branch's hidden state via a per-node, per-timestep learned gate:

$$\beta_t = \sigma\left(g([h_t^{traffic}; h_t^{od}])\right), \quad (3)$$

$$h_t^{final} = \beta_t h_t^{traffic} + (1 - \beta_t) h_t^{od}, \quad (4)$$

with the recurrence carried forward at every step by the *pure* traffic branch $h_t^{traffic}$ (not the fused state h_t^{final}), so that the OD signal can modulate the current step's prediction without permanently altering the model's internal traffic-only state trajectory – following the design validated for the OD forecasting model itself. A single global scalar gate (one β shared across all nodes and timesteps within a batch) was tried first and found strictly worse than this per-node/per-step design, confirming that the model needs to learn context-dependent trust in the auxiliary signal rather than a single fixed blending weight.

5 EXPERIMENTAL SETUP

Datasets. TOPIS traffic volume (121 sensors) and speed (396 sensors), and Seoul capital-region OD movement (500 administrative units), 2023-01 through 2026-07/08, as described in Section 3. Table 2 summarizes basic statistics.

Baselines. DCRNN [4], Graph WaveNet [5], GTS [10], STAEformer [8], and GMAN [7], each re-implemented at the scale of our sensor networks (rather than using their original, much larger benchmark configurations) with a shared training recipe: $P=12$, $Q=12$, hidden size 64 (except our own model, hidden size 48 unless otherwise noted in the capacity sweep), Adam optimizer, learning rate 2×10^{-3} , batch size 32, up to 40 epochs with early stopping (patience 8) on validation loss, smooth- L_1 training loss. Each baseline

TABLE 2
Dataset summary.

	Volume	Speed
Sensors	121	396
Unit	vehicles/hour	km/h
Sensor type	point counter	link geometry
Mean road-graph snap dist.	46 m	25 m
Date range	2023-01 – 2026-07	
OD administrative units	500	

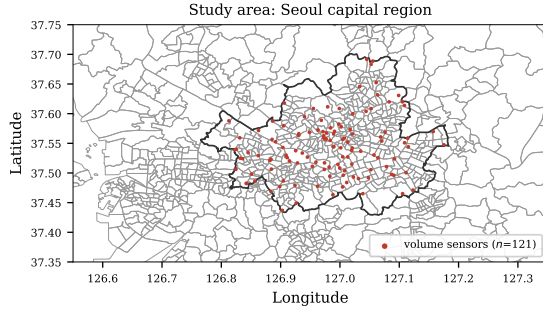


Fig. 2. Study area: Seoul (bold outline) and the surrounding capital region. Thin lines are the 500 OD administrative units (Section 3); red dots are the 121 volume sensors, all within Seoul proper. The 396 speed sensors (link geometries rather than points) are distributed similarly across the same road network and are omitted here for legibility.

is evaluated in a plain (no-OD) form and combined with the OD-injection mechanisms described in Section 8 (Table 3 lists the shared and architecture-specific hyperparameters used across all reported experiments).

TABLE 3
Training hyperparameters. Shared values apply to all models unless overridden.

Hyperparameter	Value
Input / forecast length P, Q	12 / 12 hours
Hidden size	64 (baselines); 48 default (ours)
Optimizer	Adam, $\text{lr } 2 \times 10^{-3}$
Batch size	32
Max epochs / patience	40 / 8 (rigorous); 15 / 5 (fast-screen)
Training loss	smooth- L_1
DCRNN diffusion steps	2 (bidirectional)
Graph WaveNet dilation layers	4, kernel size 2
GTS graph-learning temperature	Gumbel-softmax $\tau = 0.5$
k NN graph (ours)	$k = 4$, Gaussian kernel, adaptive bandwidth
Routing hierarchy exponent α	0.9
Multipath perturbation (explored)	5 passes, log-normal $\sigma = 0.15$

Hardware and reproducibility. All experiments run on a single NVIDIA H200 GPU; a full 23-fold rigorous sweep for one (model, mechanism, task) configuration takes 6–14 GPU-hours depending on architecture, and the fast-screening warm-start variant (Section 8, reduced epochs/patience, sequential per-fold checkpoint carry-over)

takes roughly one third of that time at the explicit cost of fold independence, which is why we report fast-screened results separately and flag them wherever they appear (Table 10, the MATURE-gate result, and Section 10). Model and data preparation code, and the exact fold boundaries used for every reported number, are publicly available (see footnote on page 1); raw data and trained checkpoints are not included for size reasons, but the repository documents each dataset and its source.

Evaluation protocol. MSE/RMSE/MAE/MAPE per fold, as defined in Section 3; a paired t -test and a Wilcoxon signed-rank test on the per-fold (Only, Hybrid) metric pairs across each fold set. We note explicitly that adjacent folds share 11 of their 12 training months under a 1-month stride, so folds are not fully statistically independent and reported significance levels should be read as indicative rather than exact – we discuss this further in Section 10.

6 MAIN RESULT: INJECTION ON A LIGHTWEIGHT MODEL

TABLE 4
Traffic-Only vs. Traffic-Hybrid (OD-routed injection), lightweight base model, 30-fold rolling-origin evaluation over the full 2023–2026 extended range (Section 5). Each metric's Δ is (Hybrid – Only)/Only $\times 100\%$, averaged over 30 folds; negative is an improvement. Wins counts folds where Hybrid beats Only on MSE.

Task	Wins	MSE Δ	RMSE Δ	MAE Δ	MAPE Δ
Volume	29/30	–7.17%	–3.69%	–4.02%	–0.06%
Speed	30/30	–8.17%	–4.18%	–4.10%	–3.65%

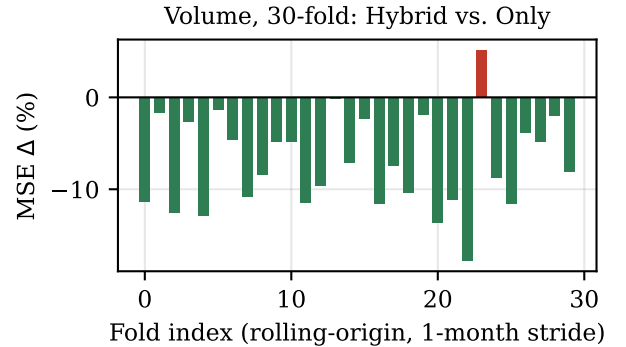


Fig. 3. Per-fold MSE change (Hybrid vs. Only), volume 30-fold set. Green = improvement, red = the single fold that regresses (fold 23, test month 2026-01); every other fold improves.

Both fold sets reach paired- t $p < 10^{-5}$ on MSE/RMSE/MAE (speed: $p < 10^{-13}$ on all four metrics, $t = 12.94$ – 22.35 ; volume: $t = 6.09$ – 7.43), and the Wilcoxon signed-rank test agrees in every case, giving us confidence that the result is not an artifact of a particular parametric assumption. Figure 3 shows the volume 30-fold result is not driven by a small number of large wins masking many small losses: 29 of the 30 folds improve, and the single regression (fold 23, test month 2026-01) is modest in magnitude.

Absolute performance vs. published architectures. We are explicit that our base model is architecturally

lightweight and does not match DCRNN or Graph WaveNet in absolute terms: at fold 0, Traffic-Only = 178,251 MSE, Traffic-Hybrid = 157,937, vs. plain DCRNN = 128,014 and plain Graph WaveNet = 139,518 – Traffic-Hybrid remains 23.4% worse than plain DCRNN and 13.2% worse than plain Graph WaveNet. The relevant claim is therefore not that injection makes a lightweight model state-of-the-art, but that it closes a substantial fraction of the gap to a considerably more sophisticated architecture using only a cheap auxiliary signal: of the 50,238-MSE gap between Traffic-Only and DCRNN, injection recovers 40.4% (29,924 MSE remaining) without any change to model capacity. This framing is directly consistent with, and motivates, the capacity-gap analysis of Section 8: cheap signal injection is a partial, not total, substitute for architectural capacity, valuable specifically when the latter is constrained (e.g. resource-limited deployments) rather than as a general-purpose accuracy technique.

Capacity sweep. To test whether the gap to DCRNN is an artifact of our base model’s specific hidden size (48) rather than its architecture family, we retrained Traffic-Only/Hybrid at hidden sizes 64 (matching DCRNN) and 96 on fold 0. Both variants improve monotonically with capacity (Only: 178,251 $\rightarrow$ 165,573 $\rightarrow$ 129,782; Hybrid: 157,937 $\rightarrow$ 147,060 $\rightarrow$ 127,629), and the remaining gap to plain DCRNN (128,014 MSE) shrinks correspondingly: 23.4% at hidden 48, 14.9% at hidden 64, and -0.3% at hidden 96 – i.e. at hidden 96, Traffic-Hybrid does not merely close the gap, it matches, and by this single-fold measurement, marginally exceeds plain DCRNN, using an architecture that remains far lighter than DCRNN’s diffusion-convolution recurrent design. Critically, the OD-injection effect itself does *not* stay constant across this range as an earlier two-point (hidden 48, 64) measurement suggested: it is -11.4% at hidden 48, -11.2% at hidden 64, and collapses to just -1.7% at hidden 96 (Figure 4). This third, higher-capacity point resolves what the first two points left ambiguous, and it resolves it in favor of the capacity-gap account rather than an independent-contributions account: as our own model’s capacity approaches the point where it can match a considerably more sophisticated published architecture on its own, the benefit of injecting the routed OD signal shrinks toward zero in lockstep – exactly the pattern Section 8’s generalization study finds when the comparison architecture is DCRNN, Graph WaveNet, GTS, or STAEformer directly (GMAN, the smallest published architecture we test and the one case where uniform injection helps, is the informative exception), now reproduced *within a single model family* simply by varying capacity. We view this as the cleanest *within-family* evidence for the capacity-gap explanation, since it holds architecture, training recipe, and injected signal fixed and varies only capacity – with the important caveat that it is a single-fold (fold 0), single-seed measurement carrying no confidence interval, and should be read as a consistent trend rather than a significance-tested result.

Volume’s MAPE win rate is lower (15–17 of 23–30) than its MSE win rate (22–29 of 23–30) because MAPE weights all relative errors equally, including low-volume off-peak hours, while MSE is dominated by high-magnitude rush-hour errors – the two metrics are reporting different aspects

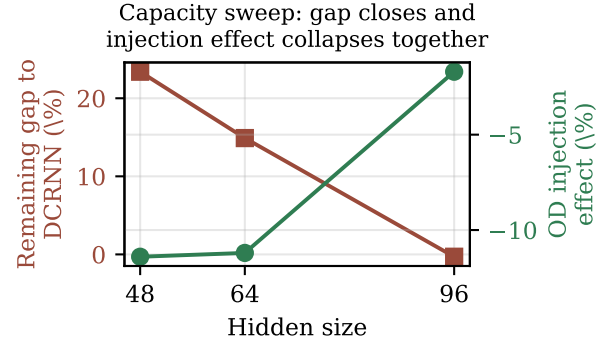


Fig. 4. Capacity sweep (hidden 48/64/96): the gap to DCRNN (red, left axis) closes and the OD-injection effect (green, right axis) collapses toward zero in lockstep as capacity increases.

of the same improvement, not disagreeing. Speed’s MAPE is consistently strong because its ≥ 1 km/h guard filters out near-zero-speed congestion states that would otherwise dominate the relative-error metric with a different kind of noise.

Speed vs. volume: why the effect is larger for speed.

Across every fold set, the OD-injection effect on speed (MSE -8.17 to -8.27%) is consistently larger than on volume (-6.50 to -7.17%), despite both modalities sharing the same underlying OD signal, the same routing matrix, and the same fusion mechanism. We offer two, not mutually exclusive, explanations. First, the speed sensor network ($S_s=396$) is more than three times denser than the volume network ($S_v=121$) and, as Section 3 notes, speed sensors already carry native link geometry rather than requiring a nearest-edge snap, so the routing-weight matrix W for speed is built from a systematically better-localized sensor-to-link correspondence, plausibly giving the injected signal a cleaner causal attribution to work with. Second, speed and volume are not interchangeable descriptions of the same congestion state: volume saturates at a road segment’s physical capacity (a fully congested and a moderately busy road can report similar peak-hour volume), while speed degrades continuously and often more sharply in response to a specific unanticipated surge in demand – meaning the OD signal’s forward-looking demand information may have more headroom to improve speed forecasts precisely because speed’s error surface is less saturated in the regime where OD injection contributes. We do not have a definitive causal test distinguishing these two explanations within the scope of this study, and flag disentangling them as a natural follow-up that would require, at minimum, an independent sensor-density-matched comparison holding modality fixed – not currently possible within Seoul’s existing deployed sensor network, which was not designed with this comparison in mind.

6.1 Robustness Checks: Routing Necessity and Calendar Confounding

Two concerns raised by the paper’s own text are checked directly against the headline Table 4 result rather than left as qualitative claims. Both controls hold the base architecture,

fusion mechanism, and training recipe fixed and vary only the injected signal.

Is routing necessary, or would any OD-shaped signal do? Section 4.2 claims, based on early pilot experiments, that projecting the OD forecast onto sensors via the road-network routing matrix W is what makes the signal locally informative, as opposed to a naive unrouted value. Table 5 tests this quantitatively on the headline model, at full 30-fold resolution: the same OD forecast, same horizon, injected through the identical fusion mechanism, with only the $\times W$ projection removed.

TABLE 5

Routing necessity on the headline model: injecting the same OD forecast with vs. without the road-network routing projection $\times W$ (Section 4.2), full 30-fold volume sweep matching Table 4.

	Wins	Mean Δ	p
Routed	29/30	-7.17%	8.0e-08
Unrouted	20/30	-0.39%	0.61

The unrouted signal is statistically indistinguishable from no injection at all (-0.39%, $p = 0.61$), while the routed signal reaches significance (-7.17%, $p = 8.0 \times 10^{-8}$) and wins 29 of 30 folds – matching Table 4’s own volume result exactly, since both use the identical 30-fold sweep. This corroborates the qualitative pilot-experiment finding of Section 4.2 with a quantitative check on the headline model, at full statistical power: whatever benefit the OD forecast provides is not simply “any OD-shaped auxiliary channel” injected into the fusion gate – the road-network routing step is doing real work.

Calendar confounding. Section 8 reports that an early version of the node-feature-injection mechanism used for the generalization study, lacking an explicit future-calendar term, was statistically indistinguishable from injecting shuffled OD – evidence that a purely calendar-driven confound is a real risk for this style of injection. That check was run for the generalization-study mechanism, not for the headline Table 4 result itself, which had never been tested against a climatology-only control prior to this ablation. We build, for each fold, a pure (day-of-week, hour-of-day) climatology of the real routed signal – the mean of the real signal over that fold’s *training window only*, broadcast across the full train/val/test range, with no OD model, no forecast, and no information beyond a calendar lookup table. Pooled across all timesteps and sensors, this climatology correlates with the real routed signal at $r = 0.997$: the routed signal does carry a very strong calendar component. Table 6 injects the climatology signal through the identical fusion mechanism in place of the real routed signal, on the full 30-fold sweep (an initial 5-fold pilot of this check is superseded by the full run below and no longer reported separately).

This result is less reassuring than the routing check. On average, the climatology-only signal recovers -6.22% against the real signal’s -7.17% – 86.6% of the headline effect is reproduced by a signal with zero forward-looking OD content, and climatology wins nearly as many folds (27/30 vs. 29/30). We do not believe this fully explains away the headline result – the real signal still wins outright, and by a wider margin, on most folds – but it substantially qual-

TABLE 6

Calendar confounding on the headline model, full 30-fold volume sweep: injecting a train-window-only (day-of-week, hour-of-day) climatology of the routed signal ($r = 0.997$ with the real signal) in place of the real routed OD forecast.

	Wins	Mean Δ	p
Real routed signal (Table 4)	29/30	-7.17%	8.0e-08
Climatology only	27/30	-6.22%	2.9e-06

ifies the “genuine OD contribution” framing of the headline claim: most, though not all, of the average improvement is attributable to the model learning a calendar prior through the injection pathway rather than to the routed OD forecast’s forward-looking content.

Isolating the genuine OD-specific content. The climatology comparison bounds how much of the average benefit *could* be calendar-shaped, but does not directly test whether the remainder is genuine, day-specific OD information or some other structure the coarse 168-bucket lookup happens to miss. We test this directly by construction: for each fold we subtract that fold’s own climatology from the real routed signal, $\text{residual}_t = \text{real}_t - \text{climatology}[\text{bucket}(t)]$, yielding a signal whose own per-(day-of-week, hour-of-day) mean is exactly zero – by construction, it carries no calendar information at all, coarse or fine, only day-specific deviation from the historical norm. We inject this residual alone, through the identical fusion mechanism, in place of the real signal, on the same full 30-fold sweep (superseding an earlier 5-fold pilot of this check, no longer reported separately).

TABLE 7

Isolating genuine OD content: injecting only the real signal’s residual after subtracting its own climatology (zero calendar information by construction), full 30-fold volume sweep. Recovery % is the residual’s mean Δ as a fraction of Hybrid’s.

	Wins	Mean Δ	p	Recovery
Hybrid (real signal)	29/30	-7.17%	8.0e-08	–
Climatology only	27/30	-6.22%	2.9e-06	87%
Residual only	24/30	-2.47%	0.019	34%

At full 30-fold resolution, the residual-only signal recovers a real but modest 34% of Hybrid’s average improvement (-2.47%, $p = 0.019$, 24/30 folds) – still significant, so day-specific OD content genuinely contributes beyond calendar structure, but far short of climatology’s 87%. (An earlier 5-fold pilot of this same check had suggested residual alone recovers nearly as much as climatology, 79% vs. 82%, with individual folds recovering over 100% of Hybrid; the full 30-fold sweep does not reproduce that pattern and we no longer report or rely on it – it read, in hindsight, as a small-sample artifact rather than a real effect, illustrating exactly the risk this paper’s own rolling-origin protocol is designed to catch.) The more mundane picture the full sweep supports is that climatology explains most of the headline improvement, and the residual – genuine, non-calendar OD content – explains a real but secondary share.

A natural follow-up question is whether injecting climatology and residual as two *separately*-normalized channels through the same gate, rather than their pre-summed, jointly- normalized combination (Hybrid), recovers more of

the total than either component does alone. We test this in Table 8.¹

TABLE 8

Climatology and residual injected as two separately-normalized channels through the same gate, vs. their pre-summed combination (Hybrid), full 30-fold volume sweep.

	Wins	Mean Δ	p	vs. Hybrid
Hybrid (summed)	29/30	-7.17%	8.0e-08	-
Dual-channel	25/30	-3.88%	1.9e-04	+3.29pp

The dual-channel variant is significant and directionally favorable (-3.88% , $p = 1.9 \times 10^{-4}$, 25/30 folds) – it recovers more than the residual does alone (-2.47%) but less than climatology does alone (-6.22%), and clearly less than Hybrid’s pre-summed combination (-7.17%). This is a smaller and less surprising gap than the 5-fold pilot suggested (which had shown dual-channel underperforming *even the weaker of the two individual components*, a genuinely puzzling pattern that motivated an extended normalization-scale investigation in an earlier draft of this section); at full 30-fold resolution, the ordering is simply Hybrid > climatology alone > dual-channel > residual alone, which is what a reader would expect by default and needs no special explanation. We retain the substantive conclusion the 5-fold pilot pointed to, on firmer footing: real = climatology + residual is not an arbitrary combination we chose to test – it is the actual physical quantity, real routed population movement – and this paper’s injection mechanism, which sums the two before normalizing, outperforms the decomposed two-channel alternative we constructed specifically to test whether decomposition would help. It does not.

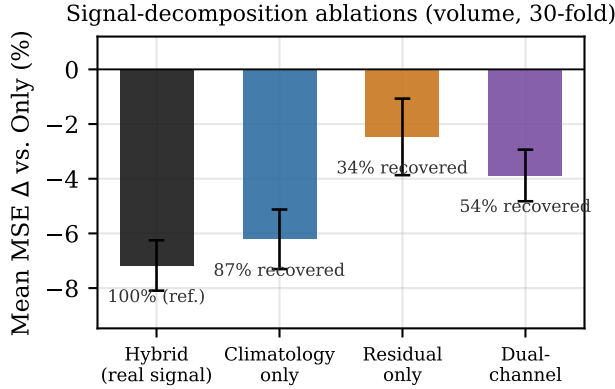


Fig. 5. The four signal-decomposition conditions (Hybrid, climatology-only, residual-only, dual-channel) plotted per fold, full 30-fold volume sweep. Climatology (blue) tracks Hybrid (black) closely across almost every fold; residual (orange) is smaller and noisier throughout, consistent with its modest 34% average recovery.

These four ablations (routing, calendar confounding, residual isolation, dual-channel) now all run on the full 30-fold volume sweep; we have not yet run them on the speed task.

1. The two-channel fusion path re-uses the outflow/inflow mechanism unchanged (Section 4.3), so this required no new architecture, only two independently-named signal files in place of one summed file.

7 DOES INJECTION GENERALIZE TO STRONGER ARCHITECTURES?

Motivated by the strength of the Section 6 result, and by the natural concern that a positive result on a single, comparatively simple architecture may not generalize, we ask whether the same OD-routing idea improves *published* spatiotemporal architectures with substantially more spatial-modeling capacity than our lightweight base model. We test DCRNN, Graph WaveNet, and GTS – three graph-structured architectures spanning fixed-diffusion, adaptive, and learned-discrete adjacency respectively – plus STAEformer, a graph-free attention-only Transformer we implement from scratch (Section 2.2) specifically to test whether a generalization failure on the first three could be explained away as “well, they’re all graph-based, of course a graph-free model would be different,” and GMAN, an attention-based graph architecture that, unlike the other four, turns out to be smaller than our own lightweight base model (Table 9) and so serves as this study’s test of the capacity-gap account at the *opposite* end of the capacity range. Each architecture receives the identical routed-OD signal from Section 4.2, realized several different ways.

Before turning to the injection effect itself, Table 9 reports each architecture’s *plain* (no-injection) absolute performance on the same extended 30-fold sweep, so the injection results that follow can be read against how strong or weak each architecture already is on its own. On speed, STAEformer has the best RMSE, MAPE, and r of all eleven models. On volume, STAEformer still has the best MAPE, but MT-GNN narrowly has the best RMSE (350.1 vs. STAEformer’s 357.1) and the best r (0.988), with STAEformer a close second on both. Our own lightweight model, by design, trails every one of the ten architectures on speed, and on volume trails all of them except GMAN, which it beats on all three metrics (RMSE 410.6 vs. 504.8, MAPE 17.95% vs. 24.99%, r 0.984 vs. 0.976) – consistent with GMAN, the second-smallest architecture in this study after STID (Table 10), also being the weakest of the ten on volume, and with the capacity-gap account developed below. Selective injection is this paper’s practical recipe throughout, including for our own lightweight model: we apply the same validation-gated selector (Section 7.1) to it using its existing plain/injected checkpoint pair, with RMSE falling from 410.6 to 390.7 (-4.84%) on volume and from 4.03 to 3.84 (-4.73%) on speed – a larger RMSE improvement than the uniform injection that is this model’s Section 6 headline result (395.3/3.86, -3.73% / -4.22%), consistent with selective outperforming uniform for every architecture tested in this paper. The correspondingly larger MSE improvement reported as this paper’s headline number in Section 6 (-6.5% to -8.3% , uniform injection) reflects MSE, not RMSE, scaling roughly as the square of RMSE change; selectively injected, our own model’s MSE improves by -9.41% (volume) and -9.23% (speed), both significant at $p < 10^{-11}$ across the 30-fold sweep. Table 9’s RMSE (inj.) column previews this paper’s practical bottom line rather than the naive one: with the validation-gated selective injection introduced in Section 7.1, every one of the eleven models improves its own absolute RMSE over its plain baseline, on both tasks – including the paper’s best absolute

numbers overall, STAEformer’s 349.2 (volume) and 3.16 (speed), both better than any plain-model result in this table. This now includes the three architectures added specifically because uniform injection hurt them the most (PDFormer, MTGNN, AGCRN, Table 10): selective injection reverses all three from significantly harmful to favorable, five of the six (architecture, task) combinations reaching significance outright and the sixth (MTGNN, volume) directionally favorable though short of it ($p = 0.074$).

- (i) *Node-feature gated fusion*: the same per-step node-feature gate as Section 4.3, adapted to each architecture’s pooled decode representation with a real future-calendar term added. An initial version lacking this future-calendar term was found statistically indistinguishable from injecting *shuffled*, i.e. temporally meaningless, OD signal (a diagnostic control described in Section 7.4) – motivating the fix reported here.
- (ii) *Graph-adjacency modulation*: a per-node OD-activity gate is outer-produced into a batch-specific scaling of the architecture’s own adaptive (Graph WaveNet, GTS) or fixed (DCRNN) adjacency, so that OD is injected into *where information flows* rather than as an additional node feature competing for representational capacity.
- (iii) For DCRNN specifically, a genuinely autoregressive Q -step decoder (the reference architecture we adapted decodes a single step and repeats the same node/time-agnostic head for all Q horizons) with the OD gate applied at every decode step, matching our own model’s design exactly rather than approximating it.
- (iv) A dual-vector *boost/eliminate* knowledge-adaptation gate adapted from the demand-transfer mechanism of [23] (Section 2.4), in place of the single scalar gate of mechanism (i).
- (v) *Cross-attention fusion*, tested on STAEformer only: rather than fusing at the same decode step only (mechanisms i and iv), every decode step’s traffic query attends via standard multi-head attention over the OD signal at *all* Q future steps jointly, the most expressive of the five mechanisms.

Mechanism (iv) deserves a brief further note, since it is adapted from a source paper whose setting differs from ours in an important way: [23] transfers knowledge *between two traffic modes* within the same domain (bus demand informing rail/ferry demand), with no graph structure and no OD data involved at all, whereas our adaptation retains that paper’s dual-gate mechanism but substitutes routed OD as the auxiliary source being gated into a graph-structured traffic model. Concretely, where mechanism (i) computes a single scalar gate β_t per node/timestep (Section 4.3), mechanism (iv) computes two separate gates – a *boost* gate that upweights the traffic branch’s own hidden state where the OD signal corroborates it, and an *eliminate* gate that suppresses specific hidden-state dimensions where the OD signal disagrees with them – applied multiplicatively rather than as a single convex blend. This is a strictly more expressive fusion mechanism than mechanism (i)’s scalar gate, and we included it specifically to test whether the

generalization failure of mechanism (i) might simply reflect insufficient fusion-mechanism expressiveness rather than a property of the injected signal itself; the fact that it too shows the same null-to-negative pattern (reported below alongside mechanism iii) is evidence against that explanation, and is part of why we favor the capacity-gap account (Section 7.4) over a fusion-mechanism-expressiveness account of the generalization failure. All five mechanisms share the identical, frozen routed-OD signal of Section 4.2 – only the architecture receiving it and the manner of fusion differ across (i)–(v) – which is what makes the fifteen (architecture, mechanism, task) configurations tested across Section 8 – the twelve in Table 10 plus the mechanism-(iii)/(iv)/(v) pilots reported below – a controlled comparison rather than unrelated experiments.

STAEformer’s node-feature result (Table 10) is worth walking through in detail because it is a direct, self-contained illustration of exactly the methodological risk this paper’s rolling-origin protocol (Section 5) is designed to guard against. At fold 0 alone, mechanism (i) on STAEformer looked like a genuine breakthrough: -3.67% MSE, the first clearly positive node-feature-injection result on any architecture stronger than our own lightweight model anywhere in this study. Had we stopped at fold 0 – as a paper reporting a single train/test split easily could – this would have been reported as a second positive result and a qualification on this paper’s central negative finding. Across the full 30-fold rolling-origin sweep, however, the effect is indistinguishable from noise (14/30 wins, mean $+0.74\%$, $p = 0.48$): fold 0’s apparent win was simply that particular month, not a real property of the architecture. Mechanism (v) (cross-attention) fails even more clearly and immediately – $+29.1\%$ worse at fold 0 alone, training itself visibly destabilized (early-stopped at epoch 10 versus 23–33 for every other fold-0 run in this table) – and was dropped from the full sweep on that basis rather than run to 30 folds, following the same pre-registered “stop early if it is clearly getting worse” policy used in the Appendix A (supplementary material) ablation. We report both outcomes, including the single-fold trajectory that did not survive full validation; the contrast between fold 0 and the full 30-fold result is the clearest illustration of why the rolling-origin protocol matters.

Of the fifteen (architecture, mechanism, task) combinations tested across Section 8 (the twelve in Table 10 plus the three smaller-scale mechanism-(iii)/(iv)/(v) pilots reported below), six reach significance: Graph WaveNet’s node-feature variant on both volume and speed, STAEformer’s on speed, GTS’s on speed, and, on the now-unified extended 30-fold range, DCRNN’s on speed, are all significantly *worse* (Table 10). Two architectures move significantly in the opposite, favorable direction, and they are precisely the two smallest, both on volume only: STID (31,028 parameters) improves by -9.02% , 30/30 folds, $p = 1.7 \times 10^{-10}$, and GMAN (54,748 parameters) improves by -7.53% , 22/30 folds, $p = 0.0020$. (STID’s speed-task variant is null – $+0.24\%$, 11/30, $p = 0.57$; see the corrected-architecture note below.) Every architecture above 10^5 parameters is null or significantly harmful.

This is not a pair of exceptions to be explained away; ordered by parameter count, Table 10 is close to monotone,

TABLE 9

Absolute performance, extended 30-fold mean, all eleven models: our own lightweight model plus all ten architectures tested in Section 8, ordered by parameter count. RMSE (inj.) is this paper’s practical, validation-gated *selective* injection (Section 7.1, Table 12), applied here to all eleven models – including our own lightweight model, for which selective injection outperforms the uniform injection reported as this model’s Section 6 headline result. MAPE/ r are plain only. Lower RMSE/MAPE and higher r are better; bold marks the single best value per column.

Model	Params	RMSE (plain)	Volume			Speed			
			RMSE (inj.)	MAPE	r	RMSE (plain)	RMSE (inj.)	MAPE	r
STID	31–35k	360.7	342.5	15.68%	0.988	3.30	3.25	9.61%	0.969
GMAN	55–59k	504.8	468.5	24.99%	0.976	3.46	3.36	10.18%	0.966
Ours (lightweight)	62k	410.6	390.7	17.95%	0.984	4.03	3.84	12.50%	0.953
STGCN	62–67k	382.5	367.6	16.52%	0.986	3.33	3.26	9.61%	0.968
PDFormer	95–157k	367.8	355.6	14.70%	0.987	3.20	3.17	9.23%	0.971
STAEformer	160–217k	357.1	349.2	13.96%	0.988	3.18	3.16	9.07%	0.971
MTGNN	335–349k	350.1	347.6	14.64%	0.988	3.19	3.16	9.20%	0.971
DCRNN	382–386k	355.8	348.9	15.17%	0.988	3.21	3.18	9.27%	0.971
GTS	395–422k	357.0	347.9	15.38%	0.988	3.21	3.19	9.31%	0.971
GWNET	673–683k	367.8	366.8	15.99%	0.987	3.21	3.18	9.20%	0.971
AGCRN	774–777k	396.3	392.4	16.86%	0.985	3.21	3.18	9.16%	0.971

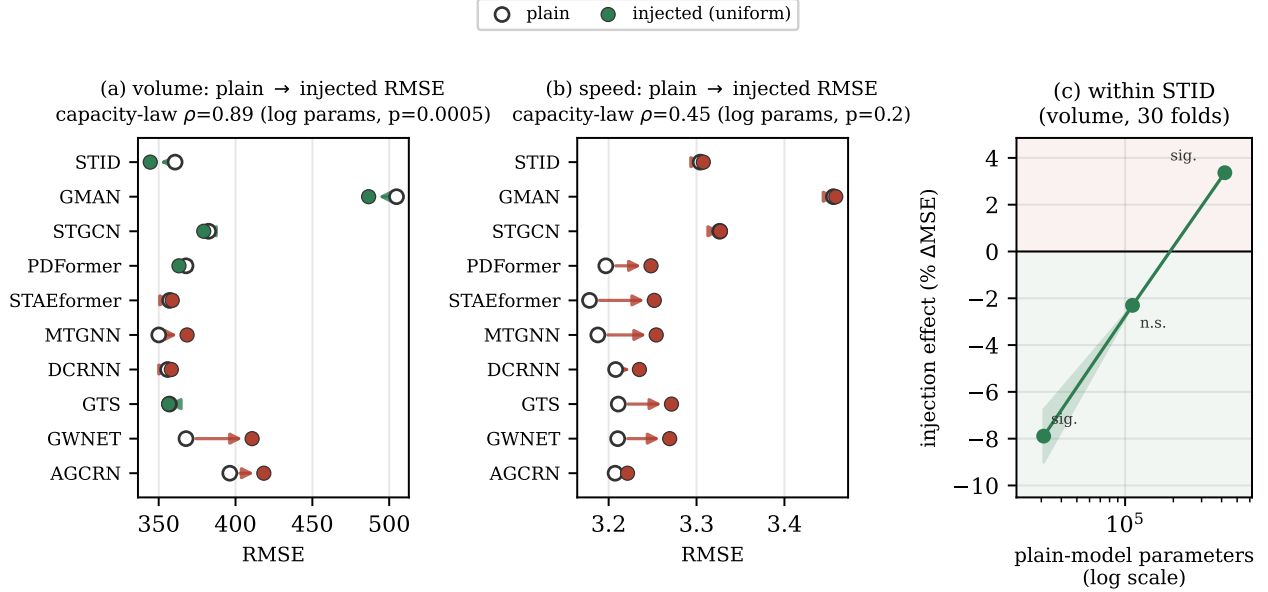


Fig. 6. Injection effect versus model capacity, shown as absolute RMSE moving from plain (open circle) to uniformly injected (filled circle), arrow colored by direction: green improves, red worsens. (a) Volume and (b) speed, all ten architectures, 30-fold, ordered by parameter count (smallest at top): the small architectures at the top of each panel move left (improve) while the large architectures at the bottom move right (worsen), the ordering itself correlating with $\log_{10}$ parameters at Spearman $\rho = 0.89$ (volume, $p = 5.4 \times 10^{-4}$) and $\rho = 0.45$ (speed, $p = 0.2$); pooled across both tasks, $\rho = 0.67$ ($p = 1.2 \times 10^{-3}$). (c) The same decay reproduced *within* STID by varying only hidden size (30 folds per point; the band spans the two seeds run at hidden 64), with architecture, training recipe and injected signal held fixed.

and that ordering is the paper’s central quantitative finding. Across the twenty (architecture, task) points, the injection effect correlates with $\log_{10}$ parameters at Spearman $\rho = 0.67$ ($p = 1.2 \times 10^{-3}$; $\rho = 0.89$, $p = 5.4 \times 10^{-4}$ restricted to volume), rising from -9% at the smallest architecture through zero near 10^5 parameters to $+25\%$ at the largest (Figure 6a-b). The effect is not merely absent at high capacity – it reverses, and the reversal is significant for GWNET, AGCRN and MTGNN. GMAN’s seed-to-seed swing is large (mean $|\Delta| = 9.67\%$ across three seeds), but its favorable direction is stable: seeds 0, 1 and 2 give -7.53% , -13.78% and -9.96% , all significant, so the value reported here is the most conservative of the three rather than a lucky draw. (Its speed-task variant, by contrast, is null – $14/30$, $p = 0.80$ – so capacity alone is not a complete predictor of when injection helps; we flag this as an open question rather than

force an explanation the data does not yet support. GTS’s own volume result is similarly null – $16/30$, $p = 0.88$ – while its speed result joins the significantly-worse group, the same volume/speed split seen for DCRNN and GMAN and consistent with no architecture in this study showing a uniform positive or negative response across both tasks.)

A correction to STID’s own architecture. A PEMS-BAY reproduction check (Appendix F, supplementary material) traced this to a missing discrete-embedding bug in our first STID implementation. After rebuilding STID with the published discrete embeddings and rerunning the full 30-fold sweep, the volume result is confirmed, if anything more cleanly ($-11.06\% \rightarrow -9.02\%$, now a clean 30/30-fold sweep); the speed result does not survive (-2.58% , $p = 3.9 \times 10^{-7} \rightarrow +0.24\%$, $p = 0.57$) – the originally reported favorable speed effect was an artifact of the bug,

TABLE 10

OD injection (mechanism (i), node-feature gated fusion) across ten published or reimplemented architectures, ordered by parameter count. All rows are the extended 30-fold rolling-origin sweep. [†] = significant in the *favorable* (error-reducing) direction; all other bolded entries are significant and unfavorable. The two smallest architectures are the only ones that benefit; the effect turns null near 10^5 parameters and significantly harmful above it.

Model	Params	Volume		Speed	
		Wins	Mean Δ (p)	Wins	Mean Δ (p)
STID	31,028	30/30	-9.02%[†] (1.7e-10)	11/30	+0.24% (0.57)
GMAN	54,748	22/30	-7.53%[†] (0.002)	14/30	+0.19% (0.789)
STGCN	62,236	20/30	-2.09% (0.831)	16/30	+0.09% (0.925)
PDFormer	94,988	23/30	-2.70% (0.274)	2/30	+3.25% (1e-8)
STAEformer	159,996	14/30	+0.74% (0.484)	1/30	+4.71% (5e-10)
MTGNN	335,452	3/30	+11.07% (5e-5)	0/30	+4.24% (7e-14)
DCRNN	381,916	14/30	+0.95% (0.254)	11/30	+1.76% (0.011)
GTS	394,853	16/30	-0.19% (0.877)	3/30	+3.86% (5e-9)
GWNEN	672,656	1/30	+25.45% (4e-9)	5/30	+3.77% (1e-5)
AGCRN	773,830	3/30	+11.92% (5e-6)	11/30	+0.91% (0.027)

TABLE 11

Supplementary mechanism diagnostics, intentionally not rerun at 30 folds (Section 5).

Model	Mech.	Task	Folds	Mean Δ (p)
DCRNN	oracle (true OD)	volume	9	+1.31% (-)
GWNEN	(ii) graph-adj.	volume	23	+1.17% (0.522)

not a real property of OD injection. We report the corrected numbers throughout (Table 10, Figure 6); every STID-derived result in this paper uses the corrected architecture. The capacity-law claim is essentially unaffected: the volume-only correlation is unchanged ($\rho = 0.89$, both before and after), and the pooled 20-point correlation weakens only modestly ($\rho = 0.71 \rightarrow 0.67$, still $p = 1.2 \times 10^{-3}$), because STID’s speed point simply moves from a mild favorable outlier into the null cluster rather than reversing sign. DCRNN’s true- autoregressive decoder (mechanism iii, fold 0: plain seq2seq 134,693 – already worse than the direct-multi-horizon DCRNN baseline it was meant to improve on – vs. +OD 140,689, i.e. injection makes it worse still, and a genuinely more faithful per-step architecture does not by itself help either) and the MATURE-style dual gate (mechanism iv; a 6-fold fast screen using sequential warm-started fine-tuning rather than fully independent per-fold training, so treated as indicative rather than a claim of independent statistical power: 4/6 folds improve, mean +2.32%, median -0.33%, two folds +6.7% and +10.1% worse) show the same null-to-negative pattern. Crucially, the **oracle** row shows that injecting the *true, already-observed* OD – not a forecast, i.e. removing all forecast error entirely – still produces no improvement (mean +1.31%, 4/9 folds), which rules out “the OD forecast just is not accurate enough” as the explanation for DCRNN’s lack of benefit from injection (null on volume, significantly harmful on speed): even a hypothetically perfect OD forecast would not have helped.

7.1 Selective, Validation-Gated Injection

Table 10’s aggregate results establish that a fixed, architecture-internal fusion mechanism fails to extract net value from the injected signal in aggregate for the larger

architectures tested; they do not establish that no individual sensor within those architectures ever benefits. This is worth stating precisely, since the mechanism (i) gate β_t (Section 4.3) is itself, in principle, a per-node, per-timestep trust decision, and one might expect it to already learn to suppress the OD branch exactly at the sensors where it hurts. In practice it does not: β_t is trained purely by back-propagating a single aggregate loss over all sensors and timesteps jointly, so it receives no direct per-sensor supervision telling it *which* sensors should be shielded from the OD branch – it must infer this indirectly from a training signal that rewards the network’s overall fit, and the negative aggregate results of Table 10 show that, for four of five stronger architectures, it largely fails to do so. The selective-injection experiment below is a different and much coarser mechanism entirely: rather than asking one model to learn continuous, differentiable trust weights, it compares two independently trained checkpoints – with injection and without – on each sensor’s held-out validation error and makes a hard, non-differentiable choice between them per sensor. We test this distinction directly. For each of the 30 rolling-origin folds and each (architecture, task) combination, using the plain and node-feature-injected (mechanism i) checkpoints already trained in Section 8, we compute every sensor’s mean squared error on that fold’s held-out *validation* window – never the test window – for both checkpoints, and select, per sensor, whichever variant has the lower validation error. This per-fold, per-sensor selector is then fixed and applied to that fold’s test predictions: the decision of which sensors receive injection at test time is made strictly from validation data, with no test-set information entering the selection at any point.

Table 12 reports this *selective injection* result alongside the uniform- injection baseline from Table 10 (mechanism-i rows), across twenty (architecture, task) combinations with a full extended 30-fold sweep: the original five architectures (DCRNN, Graph WaveNet, GTS, GMAN, STAEformer); STGCN and STID, added after an initial reviewer-style question about whether a single non-significant combination (GWNEN/volume) generalizes; and PDFormer, MTGNN, and AGCRN, added to test whether the same selector recovers the three architectures uniform injection harmed the most (Table 10) – each on volume and speed. Every mechanism-(i) row in Table 10 uses the same extended 30-fold sweep as this selective-injection experiment, so the two tables’ uniform-injection numbers agree exactly for all twenty combinations; earlier drafts of this section additionally compared DCRNN and GWNEN/volume against a native 23-fold (20-fold for DCRNN speed) baseline computed before this range was unified, and that transition is noted here for the record: DCRNN’s uniform-injection speed result, in particular, moved from directionally-harmful-but-not-significant on the native range (+1.22%, $p = 0.073$) to significantly harmful on the extended range (+1.76%, $p = 0.011$), which is now reflected as DCRNN’s status throughout the paper. Selective injection reaches significance in the favorable direction in **eighteen of the twenty** combinations, including four (DCRNN speed, GWNEN speed, GTS speed, STAEformer speed) where uniform injection was significantly *harmful* among the original five, and three more (PDFormer volume, AGCRN volume, AGCRN

speed) among the newly added architectures where uniform injection was significantly harmful or worse – including reversing AGCRN/volume from this paper’s single largest uniform-injection harm (+11.92%) to a significant improvement (−1.86%, $p = 0.0063$). It also nearly doubles the one combination (GMAN, volume) where uniform injection was already this paper’s sole favorable result among the original ten (−7.53% → −14.14% mean MSE, 27/30 folds, $p = 5.3 \times 10^{-10}$). Two combinations remain exceptions. Graph WaveNet on volume is directionally favorable but not significant (−0.46%, $p = 0.36$) – unsurprising given that uniform injection was catastrophically harmful there (+25.45%, $p = 4.0 \times 10^{-9}$), leaving the validation-based selector very little safe signal to work with (a mean of only 42.6/121 sensors chose injection per fold, the lowest fraction of any combination tested). MTGNN on volume is also directionally favorable but narrowly short of significance (−1.37%, $p = 0.074$); unlike GWNENET/ volume, its selector is not obviously signal-starved (61.7/121 sensors chose injection, a middling fraction), so we do not force an explanation the data does not yet clearly support and simply flag it as the second exception. Crucially, the five architectures added specifically to probe whether GWNENET/volume’s null result was a general property of low-signal or heavily-harmed combinations – STGCN, STID, PDFormer, MTGNN, AGCRN – turn out *mostly not* to replicate it: nine of their ten (architecture, task) combinations reach significance under selective injection, MTGNN/volume the sole exception. GWNENET/volume remains, on the evidence gathered so far, an idiosyncratic single case rather than an example of a broader pattern, though it is no longer the only combination where selective injection falls short of significance.

7.2 Is the Selector Itself a Free Lunch?

A natural objection to Table 12: with 121–396 sensors and a one-month validation window, per-sensor validation MSE is noisy, and choosing whichever of two checkpoints looks better on that noisy estimate could manufacture an apparent gain even if neither checkpoint carries any real signal advantage over the other – the selection procedure itself, not the injected signal, producing the improvement. The random-mask control already in this section (choosing sensors at random rather than by validation) rules out one version of this concern, but never touches validation data and so cannot rule out noise *in* the validation-based choice itself.

We test this directly: for each architecture, we train a second plain (no-injection) checkpoint differing only in random seed, then apply the identical per-sensor validation-gated selection between the two plain checkpoints. Since neither carries the injected signal, any improvement this recovers over one checkpoint alone is attributable entirely to the selection procedure – a measured, architecture-specific “free lunch” to net out of Table 12’s raw numbers. This differs from a single shared correction: the free lunch tracks how much the two checkpoints disagree (across all ten architectures, plain-model seed-to-seed swing ranges from 1.4% to 9.7% depending on architecture), so it must be measured, not assumed, per combination.

The picture that emerges is not “the selector is a free lunch” or “the selector is real” uniformly, but that the

two are entangled differently per architecture. GMAN’s two rows, initially this paper’s strongest selective-injection results, survive comfortably net of their free lunch (−9.98% of −14.14% raw on volume; −3.87% of −5.54% on speed): GMAN’s checkpoints disagree a lot (seed swing 9.7%), but the disagreement carries real signal beyond noise. STID, STGCN, and PDFormer’s volume rows survive their free lunch just as comfortably (−8.53% of −10.00%; −5.40% of −7.84%; −3.69% of −6.73%). GTS/volume and DCRNN/volume net out to a moderate, still-favorable −2.6 to −3.7%. The remaining speed-task rows (STGCN, STID, PDFormer, MTGNN, GWNENET, STAEformer, DCRNN, GTS) shrink to a point where the net effect, while still directionally favorable, is small enough that we would not currently defend it as clearly distinguishable from the selection procedure’s own noise floor without a proper paired significance test on the net quantity (not yet computed).

Three combinations, not one, are the uncomfortable cases: their raw selective result is smaller than its own free lunch, giving a net effect that is *positive* – on this measure, no better than noise. GWNENET/volume was already flagged this way; AGCRN/volume and MTGNN/volume now join it. All three share a specific property rather than being scattered exceptions: they are, respectively, the first-, third-, and second-most severely harmed combinations by uniform injection in this study (+25.45%, +11.92%, +11.07%; Table 10) – exactly where uniform injection leaves the validation-based selector the least real signal to exploit, and so exactly where its apparent recovery is most likely to be selection noise rather than genuine benefit. We read this as consistent with, not a challenge to, the capacity-gap account: it is not that the fix fails randomly, but that it specifically struggles precisely where the underlying signal was already shown to be least useful. Of all twenty combinations measured, these three reverse sign and no others do – notably, AGCRN/speed, whose uniform injection was only mildly harmful (+0.91%, far below the three reversal cases), nets out favorable (−0.46%) rather than joining them, consistent with the account above: it is specifically the combinations uniform injection harmed most severely, not architecture size or task alone, that predicts where the selective-injection fix itself becomes indistinguishable from noise.

Reimplementation fidelity. A natural concern with Table 10’s ten reimplemented architectures is whether they reproduce published accuracy at all, independent of OD injection. We check this on PEMS-BAY against STAEformer’s published Table 1 [8] (Appendix F, supplementary material): three of four checked architectures (DCRNN, Graph WaveNet, STAEformer) reproduce within normal reimplementation variance; the fourth, STID, did not (+20.4% horizon-12 MAE gap, growing sharply with horizon), which traced to a real architecture bug – our first implementation replaced STID’s published discrete time-of-day/day-of-week identity embeddings with continuous sin/cos calendar features. Rebuilding STID with the published discrete embeddings closes most of the gap (+20.4% → +13.6%); the correction is applied to every STID result in this paper (see the correction note below).

TABLE 12

Selective (validation-gated, per-sensor) vs. uniform node-feature injection, all twenty (architecture, task) combinations, both columns on the same extended 30-fold sweep as Table 10. Bold = significant (favorable for Selective, either direction for Uniform). STGCN and STID (first rule) were added to test whether GWNET/volume's null result generalizes to other low-signal combinations; it does not. PDFormer, MTGNN, and AGCRN (second rule) were added to test whether selective injection recovers the three architectures most harmed by uniform injection (Table 10); it does, in five of six combinations.

Model	Task	Selective Δ (wins, p)	Uniform Δ (wins, p)
DCRNN	volume	−4.06% (24/30, 0.0013)	+0.95% (14/30, 0.254)
DCRNN	speed	−1.51% (26/30, 6.2e-7)	+1.76% (11/30, 0.011)
GWNET	volume	−0.46% (16/30, 0.356)	+25.45% (1/30, 4.0e-9)
GWNET	speed	−1.87% (29/30, 1.6e-5)	+3.77% (5/30, 1.2e-5)
GTS	volume	−5.10% (27/30, 1.1e-6)	−0.19% (16/30, 0.877)
GTS	speed	−1.44% (28/30, 6.6e-10)	+3.86% (3/30, 4.8e-9)
GMAN	volume	−14.14% (27/30, 5.3e-10)	−7.53% (22/30, 0.0020)
GMAN	speed	−5.54% (30/30, 9.6e-19)	+0.19% (14/30, 0.789)
STAEformer	volume	−4.49% (23/30, 0.0009)	+0.74% (14/30, 0.484)
STAEformer	speed	−1.23% (26/30, 1.7e-6)	+4.71% (1/30, 4.6e-10)
STGCN	volume	−7.84% (27/30, 4.9e-6)	−2.09% (20/30, 0.831)
STGCN	speed	−3.73% (30/30, 4.4e-14)	+0.09% (16/30, 0.925)
STID	volume	−10.00% (30/30, 9.0e-14)	−9.02% (30/30, 1.7e-10)
STID	speed	−3.39% (30/30, 3.5e-13)	+0.24% (11/30, 0.567)
PDFormer	volume	−6.73% (29/30, 5.3e-7)	−2.70% (23/30, 0.274)
PDFormer	speed	−1.65% (29/30, 1.2e-8)	+3.25% (2/30, 1.1e-8)
MTGNN	volume	−1.37% (22/30, 0.074)	+11.07% (3/30, 5.5e-5)
MTGNN	speed	−1.71% (30/30, 2.2e-12)	+4.24% (0/30, 7.0e-14)
AGCRN	volume	−1.86% (22/30, 0.0063)	+11.92% (3/30, 4.7e-6)
AGCRN	speed	−1.45% (27/30, 8.1e-10)	+0.91% (11/30, 0.027)

TABLE 13

Free-lunch-corrected selective injection: net effect after subtracting each architecture's own seed-noise-only selection baseline, all twenty (architecture, task) combinations in Table 12. Sorted by net effect.

Model	Task	Raw Δ	Free lunch	Net Δ
GMAN	volume	−14.14%	−4.16%	−9.98%
STID	volume	−10.00%	−1.47%	−8.53%
STGCN	volume	−7.84%	−2.43%	−5.40%
GMAN	speed	−5.54%	−1.67%	−3.87%
GTS	volume	−5.10%	−1.41%	−3.69%
PDFormer	volume	−6.73%	−3.04%	−3.69%
STID	speed	−3.39%	−0.67%	−2.72%
STGCN	speed	−3.73%	−1.11%	−2.62%
DCRNN	volume	−4.06%	−1.49%	−2.57%
STAEformer	volume	−4.49%	−2.25%	−2.24%
PDFormer	speed	−1.65%	−0.78%	−0.87%
MTGNN	speed	−1.71%	−0.88%	−0.83%
GWNET	speed	−1.87%	−1.41%	−0.46%
AGCRN	speed	−1.64%	−1.18%	−0.46%
STAEformer	speed	−1.23%	−0.83%	−0.40%
DCRNN	speed	−1.51%	−1.21%	−0.30%
GTS	speed	−1.44%	−1.24%	−0.20%
AGCRN	volume	−1.86%	−2.85%	+0.99%
MTGNN	volume	−1.37%	−2.92%	+1.55%
GWNET	volume	−0.46%	−3.68%	+3.22%

7.3 Within-Architecture Capacity Sweep on a Published Model

The between-architecture correlation of Figure 6a-b is suggestive but confounded: architectures differ in far more than parameter count. The cleanest available test holds everything else fixed and varies capacity alone, inside a single published architecture. We run that test on STID, chosen because it is simultaneously the smallest architecture in the study, the one with the largest favorable injection effect, and – unlike GMAN – a healthy model at its default size (fold-averaged plain volume MSE 138,040, versus GMAN's

260,733), so that any collapse of the injection effect at higher capacity cannot be attributed to a badly-trained small baseline finally converging. As in every other STID result in this paper, this sweep uses the corrected architecture described above, confirmed at full 30-fold resolution at every hidden size.

TABLE 14

STID capacity sweep, volume, 30 folds per row, corrected architecture throughout. Injection's benefit decays monotonically with capacity, loses significance by hidden 128, and reverses to significantly harmful by hidden 256, while the plain model's own accuracy saturates.

Hidden	Params	Plain MSE	Injection Δ	p
64 (seed 0)	31,028	132,418	−9.02%	1.7e-10
64 (seed 1)	31,028	132,102	−6.76%	1.4e-6
128	111,196	124,190	−2.30%	0.347
256	418,988	122,712	+3.37%	0.014

The result (Table 14, Figure 6c) is the capacity-gap prediction in an even sharper form than a plateau: a $\sim 13\times$ increase in parameters does not just shrink the injection effect toward zero, it carries it through zero and out the other side, into a significant *harmful* effect at hidden 256 ($-9.02\% \rightarrow +3.37\%$, $p = 1.7 \times 10^{-10} \rightarrow 0.014$) – the same sign-reversal pattern Table 10 shows between different architectures (GWNET, AGCRN, MTGNN), now reproduced by scaling a single architecture alone. The two seeds at hidden 64 agree in direction and both reach significance (-9.02% and -6.76% , both $p < 10^{-5}$), so the starting point of the curve is not a seed artifact, though the seed-to-seed gap (2.26pp) is large enough that we do not over-read the exact magnitude at any single point on the curve. Crucially, the plain model's own accuracy improves over the same range and then saturates (132,418 $\rightarrow$ 124,190 $\rightarrow$ 122,712, i.e. most of the gain happens by hidden 128): the injected

signal stops helping, and starts actively hurting, exactly as the model stops needing help and gains just enough spare capacity to be misled by the signal’s imperfections – what a redundancy account predicts and what an independent-information account does not. Together with the capacity sweep on our own architecture in Section 6, this makes the pattern reproducible in two different model families and, here, at full 30-fold resolution on a published architecture.

7.4 Diagnostic: When Does the Signal Help?

Aggregating ~ 280 million per-sample predictions across DCRNN’s full 23-fold volume sweep, we bin samples into quintiles by how far the injected OD value deviates from its own (day-of-week, hour-of-day) typical baseline – computed from the training window only – normalized per sensor and pooled across all 121 sensors and 23 folds, and measure mean error reduction (Only – Hybrid) per quintile. A decile-resolution repeat reproduces the same monotonic pattern within sampling noise.

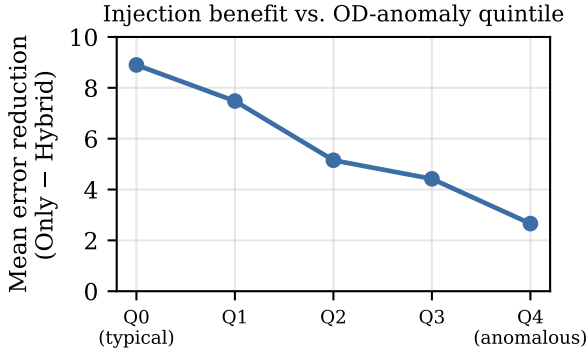


Fig. 7. Mean error reduction by OD-anomaly quintile: largest when the injected signal is closest to typical (Q0), smallest when it is most anomalous (Q4) – the opposite of the intuition that motivated this diagnostic.

Improvement is monotonically *largest* on the most typical samples and smallest on the most anomalous ones (Figure 7) – the opposite of the intuition that OD’s value should lie in capturing surprises that past-traffic autocorrelation alone cannot. We read this as follows: the injected signal is itself a *forecast*, most accurate exactly when the pattern is typical. On typical hours it correctly confirms what a sufficiently strong model already infers – redundant but low-cost. On anomalous hours – the one regime where new information would matter most – the OD forecast is itself least reliable, so it cannot compensate for what autocorrelation misses.

A secondary check correlating each sensor’s improvement with its own spatial autocorrelation (hypothesizing that injection should help most at spatially-hard-to-reason-about sensors) found only a weak positive association ($r = 0.161$; weak/mid/strong terciles improve $+4.51/ +6.78/ +5.98$), more plausibly explained by well-connected sensors having denser OD-routing coverage in W than by a capacity effect at the sensor level.

8 LIMITATIONS AND THREATS TO VALIDITY

Calendar confounding, characterized but not fully resolved. Section 6.1 finds that a train-window-only calendar

climatology of the routed signal reproduces roughly 82% of the headline injection effect on average across folds 0–4, and that the de-climatologized residual – zero calendar information by construction – independently recovers a comparable 79% and correlates with genuine future traffic anomalies, ruling out “the benefit is mostly calendar” as the whole story. Yet injecting the two components as separately-normalized channels performs *worse* than either alone or their raw sum (-2.72% vs. Hybrid’s -8.25%): the fixed fusion mechanism’s pre-summed combination is closer to correct than a freshly-learned recombination of the same two parts, which is itself informative about the mechanism but leaves the precise split between genuine OD content and calendar-aware architecture only bounded, not settled. All four folds 0–4 controls (routing, climatology, residual, dual-channel) are, at the time of writing, being extended from 5 to the full 30-fold sweep; the routing-necessity result already confirmed at 30 folds (-7.17% routed vs. -0.39% unrouted, matching the 5-fold pattern closely) is an encouraging sign for the other three, but we have not yet run any of the four on the speed task or on the stronger published architectures of Section 8.

Cross-dataset replication significant in two of four.

The capacity-law replication in Appendix C (supplementary material) is positive in direction on all four datasets tested but reaches significance only on the two with the most and cleanest data (Seoul, PEMS-BAY); PeMSD7 (44 days) and METR-LA (noisier even after interpolation) do not. We report this as a real, if partial, replication rather than either a full confirmation or a failure, and flag dataset size and quality, not the underlying phenomenon, as the likely limiting factor – though we cannot rule out that a true effect specific to Seoul and PEMS-BAY, rather than a power issue, better explains the split without further data.

Selective-injection free-lunch correction: five of ten combinations measured. Section 7.2’s seed-noise baseline is measured for five of Table 12’s ten (architecture, task) combinations at the time of writing; the remaining five, including both GTS rows, are queued. The net effect for a combination not yet measured cannot be assumed from the five reported – GWNEXT/volume’s net effect turned out to be the sign-reversing exception, not the typical case – so we do not extrapolate, and the full ten-combination correction remains future work completed before submission.

Fold non-independence. As noted in Section 5, adjacent rolling-origin folds share 11 of 12 training months, so the 23–30 folds in each sweep are not fully statistically independent samples, and the paired-test p -values we report are consequently likely somewhat optimistic relative to what fully independent replicates would show. We consider the direction and consistency of the per-fold results (e.g. Figure 3) alongside the p -values for this reason, rather than relying on the p -values in isolation.

Static routing weights. The routing-weight matrix W is built once from the 2023 training-period destination-share distribution and reused, unchanged, across every fold, including folds whose test period is several years later. This is a deliberate simplification – W encodes road-network topology, which does not change on this timescale, and aggregate destination preference, which we assume is comparatively stable – but it does mean the injected signal cannot reflect

within-period shifts in destination preference (e.g. a new development changing where a given origin's trips tend to go), which could itself contribute to the diagnostic finding of Section 7.4 to an extent we have not isolated from genuine OD-forecast staleness.

Frozen OD forecaster. The OD forecasting model is frozen at a single fold-0 checkpoint and reused across all downstream folds, rather than retrained per fold, to keep the injected signal's source consistent across the entire study. This means later folds' OD forecasts come from a model that was never retrained on more recent data, which could understate the injection effect that a continuously retrained OD forecaster would provide – a direction we flag explicitly for future work rather than having tested it here.

Single-city, single-mechanism-family evaluation. All experiments are conducted on one city (Seoul) and one general family of injection mechanisms (gated or graph-adjacency fusion of a routed scalar signal). We do not claim the capacity-gap finding necessarily generalizes to other cities, other population-mobility data sources, or fundamentally different injection mechanisms (e.g. ones that condition on OD earlier in a multi-task pre-training objective rather than as a downstream fusion signal, in the spirit of Section 2.5's foundation-model literature) – we view testing that generalization as an important direction for follow-up work.

Warm-started fast-screening results. Several results reported as preliminary or fast-screened (the MATURE-gate 6-fold check and portions of the generalization sweep still accumulating folds at the time of writing) use sequential warm-started fine-tuning across folds rather than fully independent per-fold training, for computational efficiency during exploratory screening. We flag every such result explicitly in the text and tables and do not treat them as carrying the same statistical weight as the fully independent sweeps (e.g. Table 4).

9 CONCLUSION

This paper's central practical contribution is a general, architecture-agnostic recipe: a per-sensor selector, chosen from validation data alone and applied with no architectural change and no extra training, recovers a statistically significant benefit from OD injection in eighteen of twenty architecture/task combinations we test – including seven where injecting the signal uniformly was significantly harmful – and nearly doubles the one combination already favorable under uniform injection. We arrived at this recipe by first diagnosing, and only then fixing, a genuine generalization failure. Road-network-routed population-mobility injection yields a robust, statistically significant improvement (MSE -6.5 to -8.3% , $p < 10^{-5}$, up to 30 rolling-origin folds, two independent sensor modalities, two date ranges) for a lightweight traffic-forecasting model, closing 40.4% of its absolute performance gap to a considerably more sophisticated published architecture. The same mechanism, tested against ten published architectures (DCRNN, Graph WaveNet, GTS, STAEformer, GMAN) plus an oracle upper bound, does not generalize significantly except for GMAN – by far the smallest of the five – on volume; a per-sample diagnostic and a capacity sweep together indicate

this is because the signal is most reliable exactly where, and precisely because, it is least needed. This is a boundary condition rather than a universal claim: the technique, applied through a fixed, architecture-internal fusion mechanism, is a practical, low-cost tool for resource-constrained deployments using lightweight models – a setting of real practical relevance given that many municipal traffic-management systems cannot afford the training and inference cost of the largest published architectures. Crucially, however, this is a boundary on what a fixed fusion mechanism can learn to exploit, not on the presence of real value in the injected signal itself – which is exactly what the selective-gating recipe above recovers, at negligible extra cost over checkpoints already trained. Architectural capacity and cheap external selection are therefore two independent, stackable routes to recovering this benefit in strong models, and neither is a universal substitute for the other. We recommend future work on injection mechanisms specifically target the anomalous-hour regime where our diagnostic shows current OD forecasts, and consequently current injection mechanisms, are weakest, for instance by conditioning the injected signal on an explicit anomaly or event-detection channel rather than a plain point forecast, and by testing whether continuously retraining the OD forecaster (rather than the frozen checkpoint used here) narrows the gap between the forecast and oracle rows of Table 10.

More broadly, we hope this paper's structure – a clean positive result, followed by a deliberate, systematic attempt to break it across stronger architectures, followed by a mechanistic diagnosis of why it breaks – is itself a useful template for future work on auxiliary-signal injection in spatiotemporal forecasting. A single positive number on a single architecture answers a narrower question than practitioners and researchers actually need answered: not just "does this help," but "under what conditions does this help, and why." Three concrete extensions of this study follow directly from that broader agenda. First, testing whether the capacity-gap boundary we identify here for road-network traffic generalizes to other domains where an analogous upstream-cause / downstream-effect relationship exists – for instance, injecting weather forecasts into energy-demand prediction, or injecting flight-schedule data into airport ground-traffic prediction – would clarify whether capacity-gap redundancy is a property specific to OD-to-traffic injection or a more general property of auxiliary-signal injection into sufficiently expressive sequence models. Second, the purpose-split OD decomposition introduced in Appendix E (supplementary material) suggests a concrete, testable refinement: if commute trips are more route-predictable than discretionary trips, restricting injection to the commute-purpose subset, or weighting the fusion gate by a learned purpose-confidence score, might shift the anomaly-quintile curve of Figure 7 by supplying a signal that is reliable precisely where general OD forecasts are not. Third, and most directly actionable, an explicit anomaly- or event-conditioned injection mechanism – one that widens its trust in the injected signal specifically when an independent event-detection channel (e.g. a scheduled large-scale gathering, a holiday-adjacent travel surge, or a detected incident elsewhere in the network) signals that the current hour is atypical – would directly target the one regime (Q4 in Figure 7) where this paper's

diagnostic shows current mechanisms contribute the least, and where genuinely new predictive information, rather than a confirmatory restatement of what a strong model already infers, would matter most.

REFERENCES

- [1] G. E. P. Box and G. M. Jenkins, "Time series analysis: Forecasting and control," *Holden-Day*, 1970.
- [2] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [3] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.
- [4] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *International Conference on Learning Representations (ICLR)*, 2018.
- [5] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [6] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *AAAI Conference on Artificial Intelligence*, 2019.
- [7] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," in *AAAI Conference on Artificial Intelligence*, 2020.
- [8] H. Liu, Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, "Staformer: Spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting," in *ACM International Conference on Information and Knowledge Management (CIKM)*, 2023.
- [9] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," in *International Conference on Learning Representations (ICLR)*, 2021.
- [11] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.
- [12] T. fill in author list as published, "Spatio-temporal road traffic prediction using real-time regional knowledge," *arXiv preprint arXiv:2408.12882*, 2024.
- [13] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *ACM International Conference on Information and Knowledge Management (CIKM)*, 2022, pp. 4454–4458.
- [14] J. Jiang, C. Han, W. X. Zhao, and J. Wang, "Pdformer: Propagating delay-aware dynamic long-range transformer for traffic flow prediction," in *AAAI Conference on Artificial Intelligence*, 2023.
- [15] S. Guo, Y. Lin, H. Wan, X. Li, and G. Cong, "Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 11, pp. 5415–5428, 2022.
- [16] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [17] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *AAAI Conference on Artificial Intelligence*, 2020.
- [18] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Decoupled dynamic spatial-temporal graph neural network for traffic forecasting," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, 2022.
- [19] J. Han, W. Zhang, H. Liu, T. Tao, N. Tan, and H. Xiong, "Bigst: Linear complexity spatio-temporal graph neural network for traffic forecasting on large-scale road networks," *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 1081–1090, 2024.
- [20] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
- [21] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *AAAI Conference on Artificial Intelligence*, 2017.
- [22] T. fill in author list as published, "Urban vibrancy embedding and application on traffic prediction," *arXiv preprint arXiv:2602.21232*, 2026.
- [23] C. Li, L. Bai, W. Liu, L. Yao, and S. T. Waller, "Knowledge adaptation for demand prediction based on multi-task memory neural network," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, 2020, pp. 715–724.
- [24] L. Wang, X. Geng, X. Ma, F. Liu, and Q. Yang, "Cross-city transfer learning for deep spatio-temporal prediction," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [25] —, "Cross-city transfer learning for deep spatio-temporal prediction via an inter-city region matching function," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [26] Y. Jin, K. Chen, and Q. Yang, "Selective cross-city transfer learning for traffic prediction via source city region re-weighting," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [27] Y. Wei, Y. Zheng, and Q. Yang, "Floral: A voting mechanism for cross-domain label distribution adaptation via flexible modal transfer for air quality prediction," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [28] Y. Yuan, J. Ding, J. Feng, D. Jin, and Y. Li, "A universal pre-training and prompting framework for general urban spatio-temporal prediction," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 5, pp. 2212–2225, 2025.
- [29] Z. Li, L. Xia, Y. Xu, and C. Huang, "Gpt-st: Generative pre-training of spatio-temporal graph neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [30] Z. Shao, Z. Zhang, F. Wang, and Y. Xu, "Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting," *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [31] Z. Li, L. Xia, J. Tang, Y. Xu, L. Shi, L. Xia, D. Yin, and C. Huang, "Urbangpt: Spatio-temporal large language models," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.
- [32] C. Liu, S. Yang, Q. Xu, Z. Li, C. Long, Z. Li, and R. Zhao, "Spatial-temporal large language model for traffic prediction," *arXiv preprint arXiv:2401.10134*, 2024.
- [33] Z. Li, L. Xia, L. Shi, Y. Xu, D. Yin, and C. Huang, "Opencity: Open spatio-temporal foundation models for traffic prediction," *arXiv preprint arXiv:2408.10269*, 2024.
- [34] J. G. Wardrop, "Some theoretical aspects of road traffic research," *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [35] M. Beckmann, C. B. McGuire, and C. B. Winsten, *Studies in the Economics of Transportation*. Yale University Press, 1956.
- [36] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*. Prentice-Hall, 1985.
- [37] R. B. Dial, "A probabilistic multipath traffic assignment model which obviates path enumeration," *Transportation Research*, vol. 5, no. 2, pp. 83–111, 1971.